

학교 규정집 RAG 파이프라인: Step-by-step 표

1) 전반 개요

단계	목적	주요 선택지 / 도구	비고
0. 설계	범위/요구사항 정의	Notion, Google Docs 등	어떤 규정 포함할지, 답변 스타일, 권한 범위 정의
1. 수집	규정 문서 모으기	웹에서 PDF 다운로드, 행정실 제공 파일	학칙, 대학원 규정, 장학규정, 연구윤리 등
2. 전처리	PDF → 텍스트, 구조화	<code>pypdf</code> , <code>pdfplumber</code> , <code>pytesseract</code> (스캔본)	HWP는 먼저 PDF로 변환
3. 청크	조항 단위/문단 단위 분할	직접 Python 스크립트, <code>langchain-text-splitter</code>	제1조, 제2조 단위 유지 강추
4. 임베딩	텍스트 → 벡터	OpenAI embeddings or Gemini embeddings	한국어+법령류 둘 다 잘 처리 가능한 최신 모델 사용
5. 벡터DB	검색용 벡터 인덱스 구축	FAISS	로컬/온프레임에 최적
6. RAG 파이프라인	“질문 → 검색 → LLM 답변”	LangChain / LlamaIndex / 직접 구현	retrieval + rerank + LLM 호출
7. 오케스트레이션	파이프라인 자동화, 배치 작업	LangChain(LCEL) / <code>n8n</code> / Airflow / Prefect	주기적 re-embed, 신규 문서 ingest 등
8. API/서빙	외부에서 쓸 수 있게	FastAPI / Flask / Django	<code>/ask</code> , <code>/search</code> , <code>/health</code> 등 REST API
9. UI	사용자용 챗봇 인터페이스	Streamlit, React+Next.js, Slack Bot, Kakao 챗봇	citation, 규정 조항 하이라이트
10. 모니터링/업데이트	성능 체크 & 규정 개정 반영	로그수집, 대시보드(Grafana 등)	규정 개정시 부분 재임베딩 & 재인덱싱

2) 세부 단계별 Step-by-step

A. 데이터 수집/정리

Step	상세 내용	도구/구현 포인트
A1	대상 규정 목록 만들기 (파일명, URL, 버전, 개정일)	Excel/CSV or DB 테이블
A2	웹/포털에서 PDF 일괄 다운로드	<code>requests</code> , <code>selenium</code> (필요 시), 수동 다운로드 병행

Step	상세 내용	도구/구현 포인트
A3	파일 메타데이터 정리	pandas DataFrame: doc_id, title, version_date, path

B. 텍스트 추출 & 클린업

Step	상세 내용	도구/구현 포인트
B1	PDF → 텍스트 추출	pypdf / pdftplumber
B2	스캔본 OCR	pytesseract + opencv 전처리
B3	라인 브레이크/하이픈 정리	정규식 전처리 (re.sub)
B4	장/절/조항 헤더 파싱	"제1장", "제2조(...)" 패턴으로 split

C. 청크 생성 (조항 단위)

Step	상세 내용	도구/구현 포인트
C1	기본 단위: 조항(제n조) 단위 텍스트 구성	헤더 포함한 블록으로
C2	너무 긴 조항은 하위 청크로 나누기	문단 기준 or 토큰 길이 기준 (e.g. 500 tokens)
C3	청크 메타데이터 구성	doc_id, chapter, article_no, page, version_date, tags
C4	청크 전체를 CSV/JSONL로 저장	후반 embedding 파이프라인 입력용

D. 임베딩 생성 (OpenAI / Gemini)

Step	상세 내용	도구/구현 포인트
D1	임베딩 모델 선택	- OpenAI: text-embedding-3-large/small 등 - Gemini: text-embedding-004 등
D2	배치 처리 스크립트 작성	Python + tqdm + logging (→ 진행률·에러 로깅)
D3	rate limit 고려한 호출	asyncio or sleep/retry, exponential backoff
D4	벡터 결과 + metadata를 파일/DB로 저장	np.array + parquet or 바로 FAISS로 insert
D5	임베딩 quality sanity check	유사 질문/조항 top-k 검색해보고 육안 확인

E. 벡터DB: FAISS 인덱스 구축

Step	상세 내용	도구/구현 포인트
E1	임베딩 차원 확인	OpenAI/Gemini 모델 차원 수 가져오기

Step	상세 내용	도구/구현 포인트
E2	FAISS 인덱스 선택	소규모: <code>IndexFlatIP/L2</code> 대규모: <code>IVF, HNSW</code>
E3	인덱스에 embedding insert	<code>index.add(embeddings)</code>
E4	인덱스와 metadata 동기화	별도 <code>pickle/parquet</code> 로 metadata 테이블 저장
E5	인덱스 파일 디스크 저장	<code>faiss.write_index(index, "index.faiss")</code>

F. RAG 파이프라인 (검색 + LLM)

Step	상세 내용	도구/구현 포인트
F1	쿼리 임베딩	질문을 같은 임베딩 모델로 인코딩
F2	FAISS 검색	<code>index.search(query_vec, k)</code>
F3	상위 k개 청크 + 메타데이터 정리	<code>context_blocks</code> 리스트 구성
F4	LLM 호출 프롬프트 구성	질문 + context + 시스템 지시문
F5	LLM 후보: OpenAI GPT-4.x / Gemini 2.x 등	정책상 선택, 비용·성능 트레이드오프
F6	응답 포맷 표준화	"요약", "인용 규정", "출처", "주의사항" 등 JSON or 구조화 텍스트

G. 파이프라인 관리/오케스트레이션

Step	목적	후보 도구	설명
G1	ingest 파이프라인 자동화	LangChain (LCEL), LlamaIndex , 순수 Python 스크립트	"새 문서 → chunk → embed → FAISS 업데이트"
G2	일정 기반/트리거 기반 워크플로우	n8n , Airflow, Prefect	매주/매달 정기 re-ingest, 관리자가 파일 업로드 시 트리거
G3	config 관리	<code>yaml</code> + <code>pydantic</code> or <code>dotenv</code>	모델 이름, 인덱스 경로, API 키 등
G4	로그·모니터링	<code>logging</code> , <code>prometheus</code> + <code>grafana</code>	질문 수, 실패율, 응답시간 모니터

- **LangChain/LlamaIndex:**

- RAG 체인 구성(Loader → Splitter → Embedding → VectorStore → Retriever → LLMChain)
- 쿼리 파이프라인도 한 번에 정의 가능

- **n8n:**

- no-code/low-code 워크플로우: "S3/드라이브 폴더에 파일 추가 → HTTP call로 ingest API 호출" 형태로 사용하기 좋음

H. API/서빙

Step	상세 내용	도구/구현 포인트
H1	백엔드 프레임워크 선택	FastAPI 권장 (경량·타이핑)
H2	<code>/ask</code> 엔드포인트 구현	쿼리 → embedding → FAISS → LLM → 응답
H3	<code>/health</code> , <code>/stats</code> 구현	헬스체크 + 모니터링용
H4	인증/권한 (내부용이면 간단히)	학교 VPN or SSO 연계 고려

I. UI / 챗봇 인터페이스

Step	상세 내용	도구/구현 포인트
I1	웹 UI 프로토타입	Streamlit / Gradio
I2	실제 서비스	React+Next.js or Vue + FastAPI backend
I3	UX 요소	- 질문 입력창 - 답변 + 인용 규정 하이라이트 - "원문 보기" 버튼(조항/페이지 링크)
I4	로그 수집	질문·응답·사용자 피드백 thumbs up/down 저장

J. 평가 및 운영

Step	상세 내용	도구/구현 포인트
J1	테스트셋 구성	50~100개 실제/가상의 질의 + 정답 규정 조항
J2	수동 평가	정답률, 환각률, 출처 정확도 측정
J3	로그 기반 개선	자주 틀리는 패턴 파악 → chunking/프롬프트/패널티 수정
J4	규정 개정 대응 프로세스	담당자: 새 PDF 업로드 → ingest 워크플로우 실행 → 버전 메타데이터 업데이트

3) 툴 선택 요약

기능	1차 추천	2차/대안
임베딩	OpenAI <code>text-embedding-3-*</code> or Gemini 최신 text embedding	(영어 비중 높으면) nomic/bge 로컬 모델
벡터DB	FAISS	(추후 확장) Milvus, Qdrant
RAG 프레임워크	LangChain or LlamaIndex	완전 커스텀 Python (직접 구현)

기능	1차 추천	2차/대안
워크플로우/오케스트레이션	LangChain(LCEL) + Cron / n8n	Airflow, Prefect
API 서빙	FastAPI	Flask, Django
UI	Streamlit (MVP), React (프로덕션)	Gradio